project-corebank 작업 정리

2026-11-12 · 거래 정정(역분개)·승인 절차 / 예금 상품·이자 결산(적수 방식) / 관리자 역할 분리

백엔드 **github.com/Csj930205/project-corebank** 화면 **github.com/Csj930205/project-corebank-front**

301	117	229 / 483	V16 + R
백엔드 테스트 **+33**	화면 테스트 **+21**	Postman 요청 / 검증 **+58 / +109**	DB 마이그레이션 **V14~V16 추가**

증가 폭은 11-10 마감 기준(백엔드 268, 화면 96, Postman 171/374, V13). 거래 정정은 커밋 c919e51(화면 cbcec6a), 예금 상품·이자는 861ac75(화면 9d3b2e7), 관리자 역할 분리는 이 정리를 만든 시점에 커밋 전이다. 1~4장은 거래 정정, 5~7장은 예금 상품·이자, 8~10장은 관리자 역할 분리.

1. 왜 만들었나

V12 부터 원장은 고칠 수 없다(트리거 + 앱 계정 권한). 그러면 "창구에서 10,000원을 100,000원으로 잘못 입력했다" 는 어떻게 바로잡나? README 한계에 남아 있던 이 질문의 답이 오늘 기능이다. 원장 불변성을 설명하면 면접에서 거의 반드시 따라오는 질문이기도 하다.

작업	내용
역분개 기록	원래 거래의 분개를 읽어 차변·대변만 바꿔 새 거래로 추가. 새 거래 종류 `REVERSAL_OUT` (잔액 감소)· `REVERSAL_IN` (잔액 증가), `reversal_of_id` 로 원래 거래를 가리킨다
승인 절차	정정 요청(`correction_request`) → 다른 관리자가 승인(역분개 기록) 또는 반려. 요청자 본인 결정은 코드(403)와 DB 제약 양쪽에서 거부
관리자 등록	승인자가 필요해서 `POST /admin/admins` , `GET /admin/admins` 추가
화면	"거래 정정" 탭(요청·대기 목록·승인·반려), "관리자 계정" 탭, 거래내역에 관리자용 거래 번호와 "정정 요청" 링크, 역분개 줄 표시
문서·회귀	README 두 개, Postman 폴더 "14. 거래 정정 (관리자)" 27요청

2. 흐름

단계	무슨 일이 일어나나	막히는 경우
요청	거래 번호 + 사유. 이체는 어느 쪽 거래로 요청해도 출금 쪽 거래로 맞춰 이체 전체가 대상. **원장·잔액은 그대로**	404 없는 거래 · 422 해지 지급/역분개 · 409 이미 정정됨/대기 요청 있음
승인	결정 UPDATE 를 먼저 보냄(flush) → 관련 계좌를 id 순으로 잠금 → 잔액을 반대로 움직이고 역분개 기록. 모두 한 트랜잭션	403 본인 요청 · 409 이미 결정됨 · 422 잔액 부족 · 409 해지 계좌
반려	사유 필수. 원장은 그대로. 반려 뒤에는 다시 요청할 수 있다	403 본인 요청 · 409 이미 결정됨

3. 설계에서 고민한 것 (공부 포인트)

- **분개 뒤집기 하나로 모든 종류를 처리:** 입금(차변 현금/대변 예금)의 역분개는 차변 예금/대변 현금. 이체 출금 쪽(차변 예금/대변 가계정)도 같은 규칙으로 뒤집으면 가계정 합계가 0으로 돌아온다. 종류별 분기 코드가 없다.

- **역분개에 이체 번호를 넣지 않은 이유:** 일 마감이 "이체 한 건 = 원장 거래 2건" 을 점검한다. 같은 번호를 주면 4건이 되어 정상 정정을 불일치로 잡는다.
- **두 종류의 락 (학습 가이드 10장):** 계좌는 비관적 락(`SELECT … FOR UPDATE`) — 충돌이 잦고 차례로 처리돼야 한다. 정정 요청은 `@Version` 낙관적 락 — 동시 결정은 드물고, 지면 409 로 알려 주면 끝이다.
- **왜 `saveAndFlush` 를 먼저 하나:** 변경 감지 UPDATE 는 보통 커밋 때 나간다. 그때 낙관적 락 예외가 나면 서비스 밖에서 터져 409 로 바꿀 수 없고, 역분개 INSERT 가 먼저 나가 유니크 인덱스 오류(500)가 될 수도 있다. 결정을 먼저 DB 에 보내 진 쪽이 돈을 움직이기 전에 실패하게 했다.
- **연관관계를 처음 쓴 곳 (27장):** `CorrectionRequest → LedgerTransaction` 을 `@ManyToOne(fetch = LAZY)` 로 두고 목록은 `JOIN FETCH` . @ManyToOne 은 기본이 EAGER 라 그대로 두면 N+1. ManyToOne fetch join 은 페이징이 DB 에서 그대로 되지만, fetch join 쿼리는 COUNT 를 Spring Data 가 못 만들어 `countQuery` 를 따로 적었다.
- **N+1 피하기:** 목록 화면의 이체 상대 계좌·계좌번호·역분개 id 는 요청 수와 상관없이 쿼리 3번(IN 조건)으로 모은다. 거래내역 조회와 같은 방식.
- **DB 가 마지막으로 막는 것들 (V14):** 대기 요청은 거래당 하나(함수 기반 유니크 인덱스), 역분개는 거래당 하나 (`reversal_of_id` 유니크), `decided_by <> requested_by` CHECK, 앱 계정은 결정 관련 열만 UPDATE 권한.

4. 검증

항목	결과	새로 확인한 것
백엔드 테스트 (Testcontainers Oracle)	283 통과	정정 13개: 입금·출금·이체 역분개 후 잔액=원장 합계, 가계정 0 / 본인 승인 코드·DB 양쪽 거부 / 승인과 반려 동시 5회 → 매번 하나만 성공 / 잔액 부족이면 승인 통째로 롤백 / HTTP 흐름·감사 기록. 관리자 등록 2개
화면 테스트	104 통과	본인 요청엔 결정 버튼 없음, 승인은 바뀌는 잔액을 본 뒤 확정, 반려 사유 필수, 거래내역에서 넘어오면 거래 번호 채움
Postman 회귀 (실제 서버, 개발 DB)	198 / 424 통과	두 번째 관리자를 등록·로그인해 승인, 감사 로그에 승인자·계좌 기록

만들다 찾은 것

- 화면 테스트에서 만료 시각을 2099년으로 주었더니 바로 로그아웃됐다. 자동 로그아웃 타이머 지연이 `setTimeout` 최대값(약 24.8일)을 넘으면 즉시 실행된다. 실제 토큰은 30분이라 문제없지만, 테스트는 30분 뒤로 고쳤다.
- 일계표가 거래 종류를 모두 보여 주므로 정정(차감)·정정(환입) 줄이 자동으로 생겼다(기존 테스트의 "0건" 개수 갱신).
- 감사 로그 화면에 로그인 잠금 목록·해제 작업 이름이 빠져 있던 것도 함께 채웠다.

5. 예금 상품·이자: 무엇을 만들었나

작업	내용
예금 상품	상품(이름, 연이율 %) 등록·판매 중지. 계좌는 상품 하나에 속한다(기본 상품 "기본 입출금 통장" 연 0.10%, 기존 계좌는 모두 기본 상품으로). 금리는 바꾸지 않는다: 바꾸려면 새 상품
적수 적립	매일 00:20 전날 마감 잔액 × 그날 금리를 계좌별로 한 줄씩(`interest_accrual`). INSERT … SELECT 한 문장, (계좌, 날짜) 기본키라 다시 실행해도 한 번
월 결산	매달 1일 00:30 지난달분 지급. 세전 = Σ(잔액×금리) ÷ 36,500 원 미만 버림, 소득세 14%·지방세 1.4%(각 10원 미만 버림). 계좌마다 따로 커밋

해지	해지 직전에 어제까지의 이자를 먼저 넣어 해지 지급액에 함께 나간다
화면	계좌 개설 때 상품 선택, 계좌 화면 "이자" 카드(예상 이자 세전·세금·세후, 지급 내역), 관리자 "상품·이자" 탭(상품 관리, 적립·결산 실행, 최근 지급)

6. 설계에서 고민한 것 (공부 포인트)

- **왜 적수인가:** 1만 원 × 연 2% 의 하루 이자는 0.547원. 매일 원 단위로 자르면 30일 내내 0원, 적수로 한 번에 계산하면 16원. 소수로 매일 쌓으면 반올림 오차가 쌓인다. 적수는 정수 × 소수 둘째 자리라 마지막 나눗셈 전까지 오차가 없다 (`InterestCalculatorTest`).
- **복합 분개와 일 마감 규칙 수정:** 이자 = 차변 이자비용 200 / 대변 고객 예금 180 + 대변 예수금 20. 기존 대사 규칙 "차변 = 대변 = 거래 금액" 이 이 거래를 불일치로 잡았다(전체 테스트에서 일 마감이 UNBALANCED). "차변 = 대변, 고객 예금 분개 = 거래 금액" 으로 바꿨고 모든 거래 종류에 맞는다.
- **@ManyToOne(LAZY) + @EntityGraph (27장):** `Account.product` 는 LAZY, 목록 쿼리는 `@EntityGraph(attributePaths = "product")` 로 한 번에. `AccountProductQueryCountTest` 가 Hibernate Statistics 로 SQL 문 수를 센다(1번). 첫 쪽이 쪽 크기보다 작으면 Spring Data 가 COUNT 를 생략한다는 것도 이 테스트로 확인했다. 상품 → 계좌 `@OneToMany` 는 일부러 만들지 않았다.
- **JPA 와 JdbcTemplate 섞어 쓸 때 (6장 flush):** 지급 기록(JPA)을 `saveAndFlush` 로 먼저 INSERT 해야 적수 UPDATE(JdbcTemplate)의 외래키가 그 행을 찾는다. JdbcTemplate 은 영속성 컨텍스트에만 있는 변경을 모른다.
- **대량 처리는 SQL, 계좌별 업무는 엔티티:** 적수는 계좌 수만큼 행을 쓰는 일이라 INSERT … SELECT, 지급은 잠금·잔액 규칙 (`Account.deposit`)이 필요해 엔티티로.
- **트랜잭션 범위 (28장):** 결산 전체를 한 트랜잭션으로 묶지 않고 계좌마다 `TransactionTemplate` . 실제로 잔액 한도 시험 계좌(long 최대값)가 이자를 못 받아 실패했고, 다른 계좌는 정상 지급됐다 → 테스트로 고정.

7. 예금 상품·이자 검증

항목	결과	새로 확인한 것
백엔드 (Testcontainers Oracle)	295 통과	시계를 40일 뒤로 옮겨 오늘·내일을 "끝난 날" 로: 100만 원 × 3.65% 이틀 = 세전 200 / 세금 20 / 세후 180, 3줄 분개, 두 번 결산해도 한 번, 해지 시 1,000,090원 지급, 이자 거래가 있어도 일 마감 정상, 이자는 정정 불가, 판매 중지 상품
화면	112 통과	금리 입력 규칙, 판매 중지 두 번 확인, 예상 이자·지급 내역 표시
Postman (실제 서버)	220 / 467 통과	상품 API·권한, 적립 중복 없음, 끝나지 않은 날·달 거부(422)

8. 관리자 역할 분리: 무엇을 만들었나

작업	내용
왜	관리자가 한 종류이고 관리자를 바로 만들 수 있었다. 한 사람이 계정을 하나 더 만들어 자기 정정 요청을 자기가 승인할 수 있어 4-eyes 가 이름뿐이었다(1~4장의 남은 구멍)
역할 세 개	창구 TELLER(개설·입출금·이체·해지, 정정 요청) / 책임자 APPROVER(정정·관리자 등록 승인·반려) / 운영 OPERATOR(마감, 이자, 상품, 잠금 해제, 관리자 등록 요청). 계정마다 하나, 바꿀 수 없다. 조회는 모두
계정 발급 승인	운영자가 요청(PENDING) → 책임자가 승인(ACTIVE)해야 로그인. 반려(REJECTED)된 아이디는 기록으로 남는다

처음 두 명	.env 로 운영자(ADMIN_*)와 책임자(APPROVER_*)를 만든다. 기존 관리자는 V16 에서 운영자가 된다
화면	오른쪽 위에 "창구 · 아이디". 실행 버튼은 맡은 역할에게만, 아니면 "○○ 역할만 할 수 있습니다". 관리자 계정 화면: 역할을 골라 등록 요청, 승인 대기 목록에서 승인·반려

9. 설계에서 고민한 것 (공부 포인트)

- **왜 역할을 하나만:** 한 계정이 요청과 승인을 함께 가지면 혼자서 통과한다. 하나로 묶으면 요청자와 승인자는 늘 다른 계정이고, 계정을 늘리는 일에도 두 사람이 필요하다.
- **scope 와 role 두 claim:** scope = 누구인가(ADMIN/CUSTOMER), role = 무슨 일을 맡았나. Spring Security 기본 변환기는 scope 만 읽어 `SCOPE_ADMIN` 을 만든다. 변환기를 하나 더 붙여(`DelegatingJwtGrantedAuthoritiesConverter`) role 을 `ROLE_TELLER` 로. `hasRole("TELLER")` 가 이걸 찾는다.
- **규칙은 한곳, 모르면 막는다(fail-closed):** `@PreAuthorize` 를 컨트롤러에 흩지 않고 SecurityConfig 표 하나로. 관리자 변경은 표에 없으면 `denyAll` . 새 API 에 규칙을 잊으면 아무에게도 열리지 않아 테스트에서 바로 보인다.
- **"처음 한 번은 승인 없이" 를 두지 않은 이유:** 그 예외가 곧 혼자 계정을 늘리는 틈이다. 첫 두 명은 앱 밖(.env)에서. .env 를 다루는 사람은 원래 DB 소유자 권한도 있어 새 틈이 아니다.
- **승인 대기 계정 로그인:** 상태보다 비밀번호를 먼저 비교하고, 실패는 비밀번호가 틀렸을 때와 같은 401. 응답 내용과 시간 모두로 "그 아이디가 있다" 를 알려 주지 않는다.
- **EnumType.STRING (8장 매핑):** ORDINAL 이면 enum 에 값을 끼워 넣는 순간 저장된 0·1·2 의 뜻이 바뀐다(창구가 책임자가 되는 식). 권한이라 특히 위험하다.
- **Hibernate UPDATE 는 모든 열을 보낸다 (6장 변경 감지):** 승인 때 status 만 바꿨는데 `ORA-41900` (UPDATE 권한 없음). 기본 UPDATE 는 바뀐 열이 아니라 updatable 인 모든 열을 SET 에 넣어(문장 재사용) 권한이 없는 password_hash 까지 들어갔다. 바뀌지 않는 열에 `updatable = false` . DB 권한(앱 계정은 결정 열만 UPDATE)이 이 실수를 잡아 줬다.
- **낙관적 락 재사용:** 두 책임자가 같은 등록을 동시에 결정하면 `@Version` 으로 한쪽만. 정정과 같은 `saveAndFlush` + 409 패턴.

10. 관리자 역할 분리 검증

항목	결과	새로 확인한 것
백엔드 (Testcontainers Oracle)	301 통과	변경 API 17개 × 맡지 않은 역할 = 모두 403 / 표에 없는 관리자 POST 는 모든 역할 403 / 조회는 세 역할 모두 200 / 승인 전·반려 계정 로그인 401 / 요청자 본인 결정 거부 / 역할 없는 옛 토큰은 조회만 / 승인된 창구는 계좌 개설
화면	117 통과	역할별 권한 표, 운영자가 아니면 상품 등록·판매 중지·이자 실행 없음, 창구만 정정 요청 칸, 책임자만 승인 버튼
Postman (실제 서버)	229 / 483 통과	운영자·책임자 로그인 → 창구 등록 요청 → 승인 전 로그인 401 → 승인 → 창구 로그인. 운영자 계좌 개설·책임자 입금·창구 마감은 403. 컬렉션 기본 토큰은 창구

11. 남은 한계와 다음 후보

알려진 한계

- 관리자 계정 정지·삭제·비밀번호 변경 없음. 운영자와 책임자가 공모하면 가짜 창구를 만들 수 있다(감사 로그로 사후 추적). 역할은 URL 단위(지점별 같은 데이터 권한 없음)
- 전액 정정만. 일부 금액은 전액 정정 뒤 올바른 거래를 다시 한다
- 요청자가 직접 취소하는 기능 없음(다른 관리자가 반려)
- 거래가 아닌 오류(원장 없이 바뀐 잔액 컬럼)는 정정 대상이 아니다
- 이자: 해지일·오늘 하루치는 붙지 않음, 금리 변경·우대금리·정기예금·복리 없음, 세금 납부 처리 없음, 결산은 계좌를 하나씩(대량이면 Spring Batch 청크)

다음 후보

- 정기예금(만기·중도해지 이율), 금리 변경 이력
- 이자 결산을 Spring Batch 청크로(skip·retry·재시작)
- 관리자 계정 정지·비밀번호 변경
- 리프레시 토큰, 토큰 강제 무효화
- 일 마감 분산 잠금(ShedLock), 감사 로그 outbox